项目 8 路径规划与自主导航

1、 ROS2 简介及安装

1.1、 ROS2 简介

本项目基于 ROS2 机器人操作系统，ROS 英文全称是（Robot Operating System），ROS 本身并不是一个传统的操作系统，而是安装在现在已有的操作系统上（Linux、Windows、Mac）上的软件库和工具集。

ROS2 是在 ROS 的基础上设计开发的第二代机器人操作系统，帮助我们简化机器人开发任务。与 ROS1 相比，ROS2 取消了 master 主节点。去中心化后，各个节点之间可以通过 DDS 的节点相互发现，各个节点都是平等的，且可以 1 对 1、1 对 n、n 对 n 进行互相通信。通信直接更换为 DDS 进行实现，使得 ROS2 的实时性、可靠性和连续性上都有了增强。

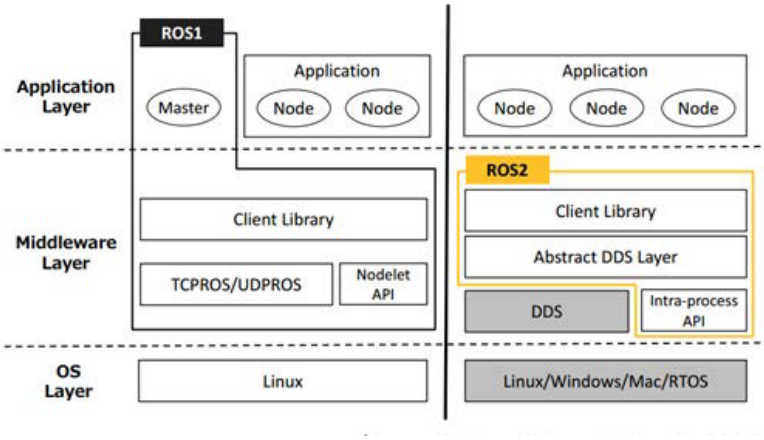


图 1-1 ROS2 架构图

1.2、 安装教程

以下安装环境基于 Ubuntu22.04, ROS2 版本为 humble.

打开终端，输入以下指令添加源。

```
echo "deb [arch=$(dpkg --print-architecture)] https://repo.huaweicloud.com/ros2/ubuntu/ $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

添加源对应的秘钥并更新。

```
sudo apt install curl gnupg2 -ycurl -s https://gitee.com/ohhuo/rosdistro/raw/master/ros.asc | sudo apt-key add -
sudo apt update
```

安装 ROS2。

```
sudo apt install ros-humble-desktop
```

本节安装的是 humble 版本的 ROS，安装目录默认在 /opt/ros/humble 下。

1.3、 节点测试

通过订阅者节点与发布者节点的通信测试,测试 ROS 是否成功安装。

在终端中启动一个订阅者节点。

```
ros2 run demo_nodes_py listener
```

在另一个终端中启动一个发布者节点。

```
ros2 run demo_nodes_cpp talker
```

观察两个节点之间是否正常通信。



图 1-2 节点通信

2、 Unitree Go2 for ROS2 SDK 简介及安装

Unitree GO2 for ROS2 SDK 是一个支持 ROS2 的 SDK，专为 Unitree GO2 系列机器人设计，包括 AIR、PRO 和 EDU 版本。这个 SDK 让开发者能够充分利用 ROS2 的强大功能,如全实时关节同步、脚力传感器反馈、Laser Scan 处理、多机器人支持以及通过 WebRTC

和 CycloneDDS 进行通信的能力。

2.1、 Go2 for ROS2 SDK 安装

首先，确保你的系统已经安装了 ROS2，并配置好相关环境。接下来，遵循以下步骤来集成 Unitree GO2 与 ROS2。

2.2、 SDK 获取

在终端中执行以下命令，在本地根目录中建立 ros2_ws 文件夹，并下载待编译 SDK。

```
mkdir -p ros2_ws
cd ros2_ws
git clone --recurse-submodules https://github.com/abizovnuralem/go2_ros2_sdk.git src
sudo apt install ros-$ROS_DISTRO-image-tools
sudo apt install ros-$ROS_DISTRO-vision-msgs
sudo apt install python3-pip clang portaudio19-dev
cd src
pip install -r requirements.txt
cd ..
```

2.3、 SDK 编译及拓展库安装

执行以下指令，加载 ROS2 环境，编译 SDK。

```
source /opt/ros/humble/setup.bash

rosdep install --from-paths src --ignore-src -r -y

colcon build
```

接下来安装相关的拓展库。

```
sudo apt update

sudo apt-get install python3-colcon-common-extensions

sudo apt install python3-rosdep2

sudo apt install ros-humble-navigation2

sudo apt install ros-humble-nav2-bringup

sudo apt install ros-humble-turtlebot3-gazebo

sudo apt install ros-$ROS_DISTRO-foxglove-bridge

sudo apt install ros-$ROS_DISTRO-tf2-sensor-msgs

sudo apt install ros-$ROS_DISTRO-slam-toolbox

sudo apt install

ros-foxglove-bridge

sudo apt install ros-humble-slam-toolbox
```

```
sudo apt install ros-humble-slam-toolbox

sudo apt install ros-humble-tf-transformations

pip install numpy==1.23.5
```

2.4、 激光点阵运算库 pointcloud_to_laserscan 安装

下载激光点阵运算库 pointcloud_to_laserscan 到本地文件夹，执行编译指令。

下载地址（选择对应 humble 的分支库）：

https://github.com/ros-perception/pointcloud_to_laserscan/tree/humble

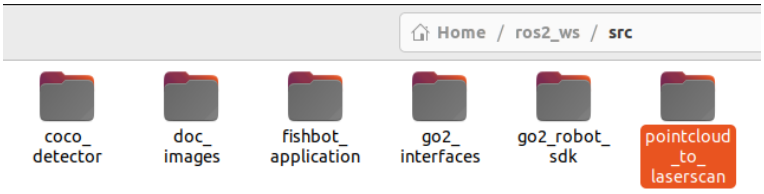


图 2-1 项目文件夹

回到上层根目录，执行编译指令。

```
colcon build
```

2.5、 配置网络连接

2.5.1、 查看 Go2 机器人网络地址

打开 Go2 机器人手机端软件，将 Go2 机器人设置为 WiFi 模式连接，点击设备/数据/整机自动检测，在 STA Network:wlan0 中找到其 IP 地址。



图 2-2 Go2 数据查看

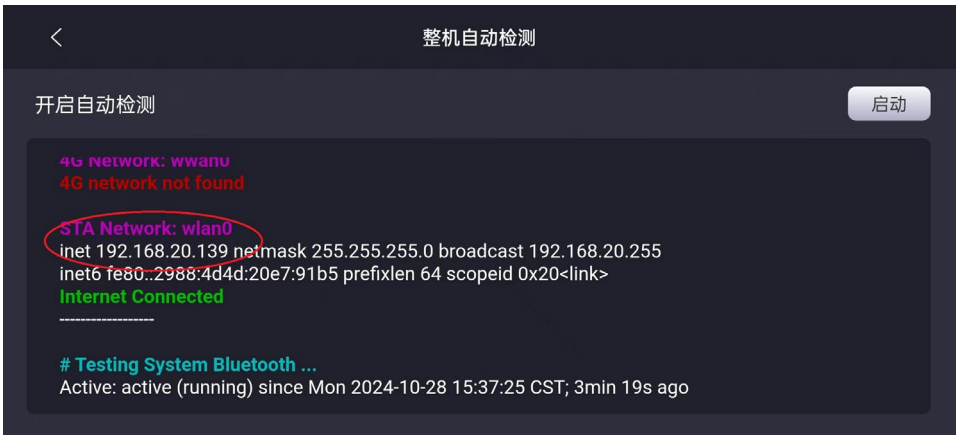


图 2-3 IP 地址查看

将开发端电脑接入 Go2 机器人的同一网络中，使用 ping 指令测试连接情况。

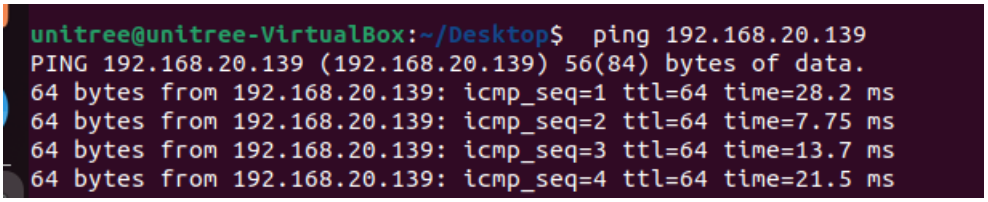


图 2-4 连接测试

2.6、 启动 ROS2 服务

Go2 机器人的手机控制程序和 ROS2 中的 Go2 服务都采用 WebRtc 协议控制 Go2 机器人，为保证 Go2 机器人的稳定运行，只能有一种应用具有主控权。

如果 Go2 机器人的手机控制程序已连接 Go2 机器人，在启动 ROS2 服务前需要先强制停止手机控制程序。

进入安装根目录 ros2_ws，通过以下命令启动相关的节点和服务。

```
source install/setup.bash

export ROBOT_IP="robot_ip"

export CONN_TYPE="webrtc"

ros2 launch go2_robot_sdk robot.launch.py
```



图 2-5 启动文件地址

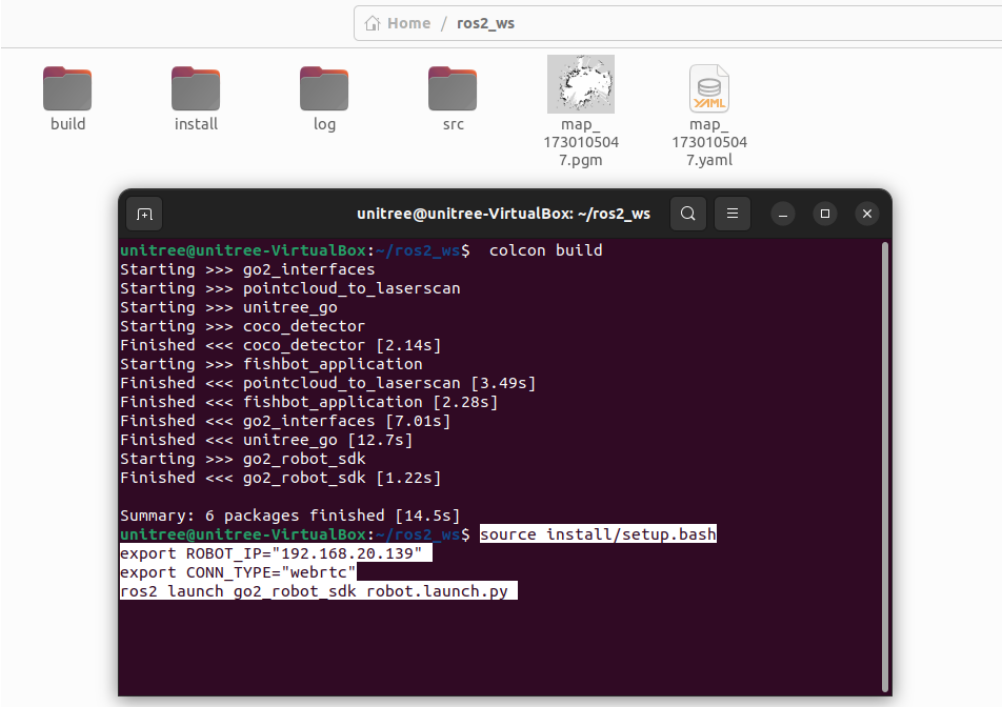


图 2-5 启动指令

上述指令启动的主要服务/节点如下所示。

① robot_state_publisher

② ros2_go2_video

③ pointcloud_to_laserscan_node

④ go2_robot_sdk/go2_driver_node

⑤ go2_robot_sdk/lidar_to_pointcloud

⑥ rviz2

⑦ Joy

⑧ slam_toolbox/online_async_launch.py

⑨ av2_bringup/navigation_launch.py

输入 ros2 node list 指令，查看当前已经启动的各节点消息。

```
unitree@unitree-VirtualBox:~/ros2_ws$  ros2 node list
WARNING: Be aware that are nodes in the graph that share an exac
intended side effects.
/amcl
/behavior_server
/bt_navigator
/bt_navigator_navigate_through_poses_rclcpp_node
/bt_navigator_navigate_to_pose_rclcpp_node
/controller_server
/foxglove_bridge
/foxglove_bridge_component_manager
/global_costmap/global_costmap
/joy_node
/lifecycle_manager_navigation
/local_costmap/local_costmap
/planner_server
/pointcloud_to_laserscan
/rviz2
/rviz2
/rviz2
/rviz_navigation_dialog_action_client
/slam_toolbox
/smoother_server
/teleop_node
/transform_listener_impl_5c2353bde8f0
/transform_listener_impl_5df2589d5b10
/transform_listener_impl_5e3daf62f3e0
/transform_listener_impl_607c275c1ec0
/transform_listener_impl_62fdd9ed0e60
/transform_listener_impl_6464675312f0
/transform_listener_impl_64dcdad64930
/twist_mux
```

图 2-6 主题查看

正常启动后，我们可以通过 RVIZ2 界面看到激光雷达数据和前端彩色摄像头数据，且机器人已经准备好接收遥控器发出的控制指令。

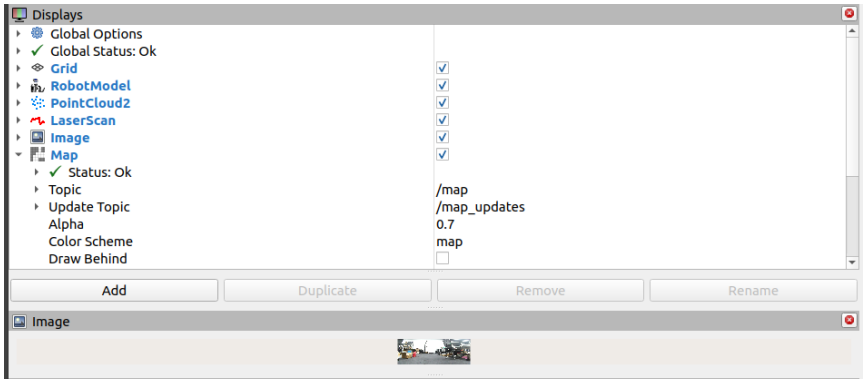


图 2-7 RVIZ2 工具栏

工具栏主要信息如下。

RobotModel 是 Go2 的当前型号。

PointCloud2 是原始 的雷达数据转换为 3D 对象/约束。

LaserScan 是转换为 x，y，z 帧之前的较低级别扫描数据。

Image 是来自前置彩色摄像头的原始数据。

Odometry 是机器狗运动的历史纪录。

以下是 slam_toolbox 生成的地图数据。在地图的基础上，还包括了 Go2 机器人模型、定位信息、运动趋势等相关数据信息。

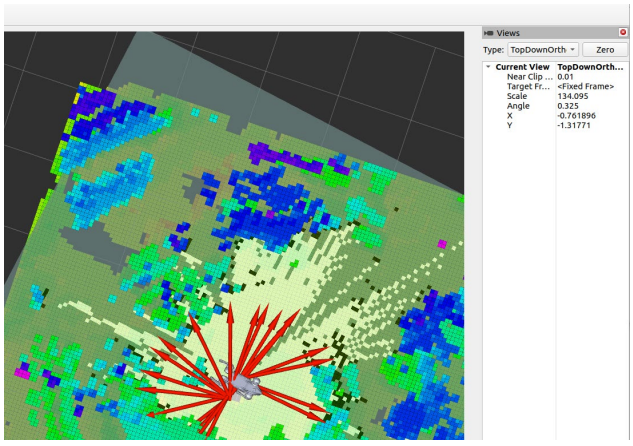


图 2-8 实时雷达数据图

2.7、 SLAM 建图

SLAM 技术利用传感器收集环境数据以实现定位和地图构建。在 ROS2 中，有多个 SLAM 功能包可供选择，例如 slam_toolbox、cartographer_ros 和 rtabmap_slam。对于二维激光雷达场景，slam_toolbox 相对容易上手，我们将使用它来创建第一张地图。

为确保建图准确，请控制机器人缓慢平稳地移动并确保建图环境地面为水平地面，起始位置距最远距离不能超过 30 米，建图时间需低于 5 分钟。

为机器人在地图上创建一个明确的起点，在屏幕左侧的菜单中选择“Start At Dock”。然后使用遥控器控制 Go2 机器人在空间中移动，我们将看到地图数据不断地进行更新和积累。在此地图中，白色、黑色和灰色像素分别表示空闲、已占用和未知空间。完成采集后，先在 “Save Map” 字段中输入文件名，然后单击 “Save Map”。然后在“Serialize Map” 字段中输入文件名，最后单击 “Serialize Map”。在当前工作路径中会看到以下 2 个新文件。SLAM 服务会订阅 map 话题来获取最新地图，然后生成一张 pgm 格式的图片和对应的 yaml 格式的描述文件。

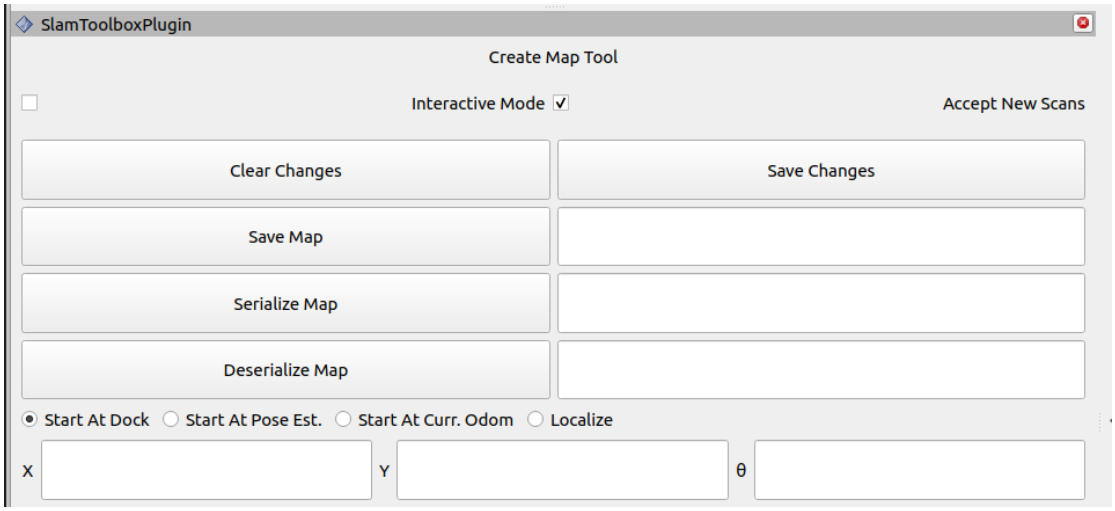


图 2-9 slam_toolbox 界面

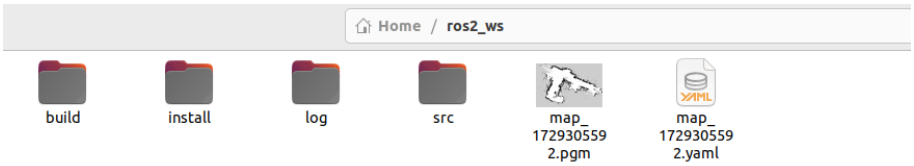


图 2-10 地图文件

在导航前需要加载已保存的地图，在 slam_toolbox 界面的 'Deserialize Map' 字段中输入地图的文件名（不带任何扩展名），然后单击 'Deserialize Map'。

重新启动并加载地图时，Go2 机器人不知道它相对于您之前创建的地图的位置。最好在原来实际的 Dock 点放置机器人并保持之前的朝向，它将具有高质量的初始位置和角度。

3、 路径规划及自主导航

地图构建完成之后，通过 Navigation2 组件的应用，可以实现 Go2 机器人的路径规划和自主导航。

3.1、 Navigation2 简介及应用

Navigation 是指机器人在环境中的路径规划和移动能力。它通常包括定位、地图构建、路径规划、避障和导航恢复行为等功能。在 ROS2 中，Navigation 功能通过 Navigation2（Nav2）实现，它是一个模块化的、基于行为树的导航框架。Nav2 使用多个独立的服务，通过 ROS2 接口（如动作服务器或服务）与行为树（BT）通信，以实现复杂的导航行为。Navigation2 的架构图如下。

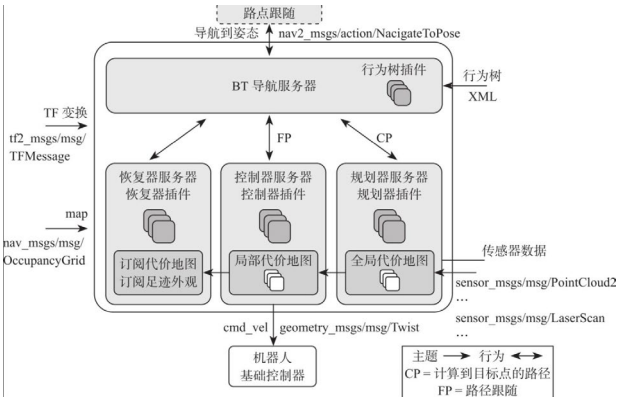


图 3-1 Navigation 2 架构图

Navigation 2 架构的核心是 BT 导航服务器，它利用 XML 行为树文件调用三个服务模块。右侧的规划器服务器负责全局路径规划，支持多种算法以提高灵活性。中间的控制器服务器根据全局路径和实时

信息完成机器人控制，同时支持多种控制器控制。

3.2、 AMCL 节点服务

为了使用代码控制自主导航，我们需要启动 AMCL 节点服务。amcl 的英文全称是 adaptive Monte Carlo localization，是蒙特卡洛定位方法的一种升级版。

Go2 for ROS2 SDK 默认未启用 AMCL 节点，参照以下步骤开启 AMCL 节点服务。

在以下路径：

/home/unitree/ros2_ws/src/go2_robot_sdk/launch/robot.launch.py 中添加 amcl 节点信息。



图 3-2 节点启动文件

```
Node(
        package='nav2_amcl',
        executable='amcl',
```

```
            name='amcl',
            output='screen',
            parameters=[{'config/amcl_params.yaml'}
            ],
        ),
```

在以下路径：

/home/unitree/ros2_ws/src/go2_robot_sdk/config/ 下添加 amcl 配置文件 amcl_config.yaml。

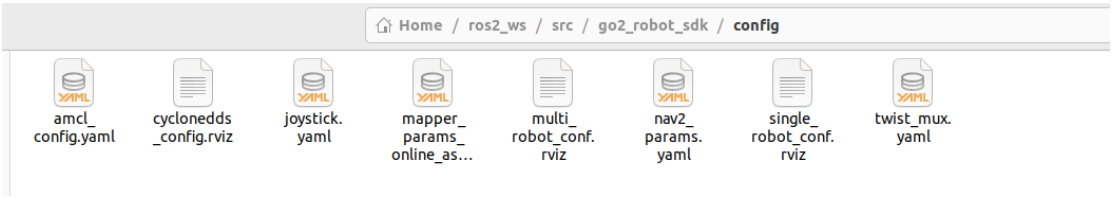


图 3-3 节点配置文件

amcl_config.yaml

```yaml
amcl:
  ros__parameters:
    alpha1: 0.2
    alpha2: 0.2
    alpha3: 0.2
    alpha4: 0.2
    alpha5: 0.2
    base_frame_id: "base_footprint"
    beam_skip_distance: 0.5
    beam_skip_error_threshold: 0.9
    beam_skip_threshold: 0.3
    do_beamskip: false
    global_frame_id: "map"
    lambda_short: 0.1
    set_initial_pose: false
    laser_likelihood_max_dist: 2.0
    laser_max_range: 100.0
```

图 3-4 amcl 参数配置

回到项目根目录 ros2_ws，运行以下指令，重新编译 SDK。

```
colcon build
```

运行以下指令，查看 amcl 节点运行情况。

```
ros2 node info /amcl
```

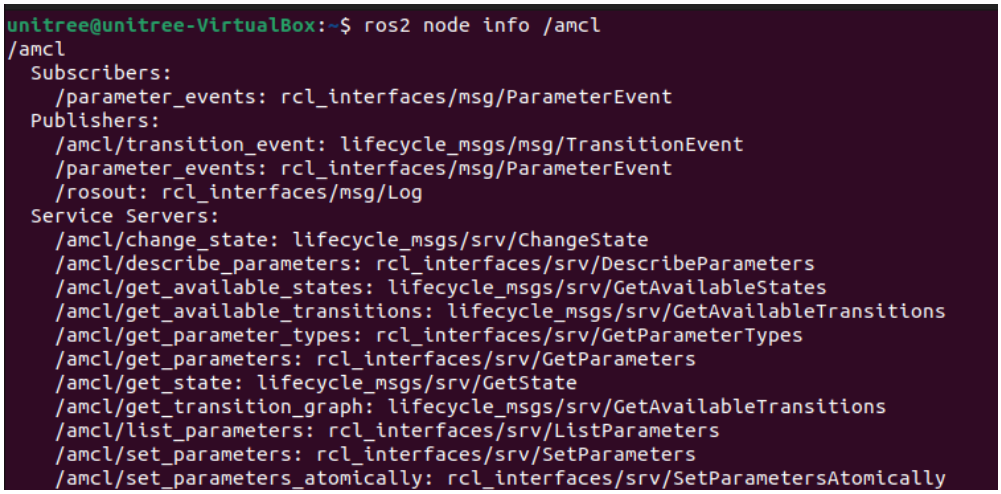


图 3-5 amcl 节点状态

3.3、 位姿获取及初始化

我们需要获取并保存机器人的实时位姿信息，以便实现初始化和自主导航。

3.3.1、 位姿获取 get_robot_pose.py

获取当前位姿信息，包括平移坐标、旋转四元数和旋转欧拉角。

```
class TFListener(Node):

    def __init__(self):
```

```
            super().__init__('tf2_listener')
            self.buffer = Buffer()
            self.listener = TransformListener(self.buffer, self)
            self.timer = self.create_timer(1, self.get_transform)
        def get_transform(self):
            try:
                tf = self.buffer.lookup_transform(
                    'map', 'base_footprint', rclpy.time.Time(seconds=0),
rclpy.time.Duration(seconds=1))
                transform = tf.transform
                rotation_euler = euler_from_quaternion([
                    transform.rotation.x,
                    transform.rotation.y,
                    transform.rotation.z,
                    transform.rotation.w
                ])
```

保存位姿信息。

```
    #将元组数据存储在文本文件中
```

```
                current_time = time.time()

                # 获取当前时间戳

                now = datetime.now()

                with open('data.txt', 'a') as file:

                # 将数据转换为字符串，并追加换行符

                    line = "平移:"+ str(transform.translation) + "旋转四元
数:" + str(transform.rotation) + "旋转欧拉角:" + str(rotation_euler)
+ "/time:" + now.strftime("%H:%M:%S") + '\n'

                # 写入文件

                    file.write(line)
```

直接运行主程序，查看实时位姿信息。

```
python3  get_robot_pose.py
```

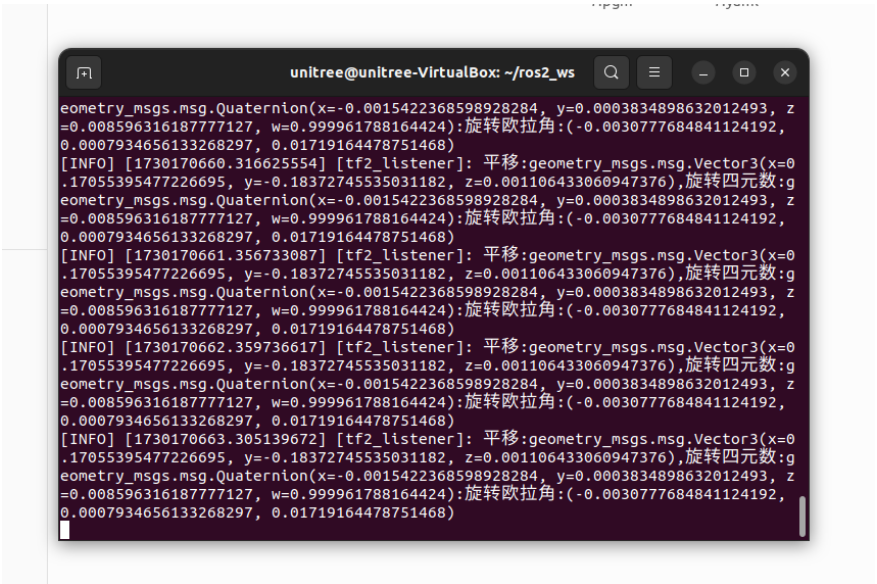


图 3-6 实时位姿信息

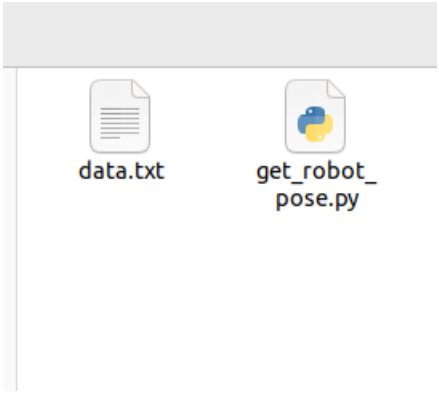


图 3-7 位姿信息保存

3.3.2、 **位姿初始化** init_robot_pose.py

```
def main():
    rclpy.init()
    navigator = BasicNavigator()
    initial_pose = PoseStamped()
    initial_pose.header.frame_id = 'map'
    initial_pose.header.stamp = navigator.get_clock().now().to_msg()
    initial_pose.pose.position.x = 0.0
    initial_pose.pose.position.y = 0.0
    initial_pose.pose.orientation.w = 1.0
```

```
    navigator.setInitialPose(initial_pose)
    #navigator.waitUntilNav2Active()
    rclpy.spin(navigator)
    rclpy.shutdown()
```

3.4、 单点导航应用

3.4.1、 菜单操作

2D Pose Estimate 是用于初始化位置的工具，选中 2D Pose Estimate，然后单击地图中机器人目前所在的大概位置，不要松开左键，拖动鼠标调整机器人朝向，如果觉得设置得不够准确，可以多次设置。

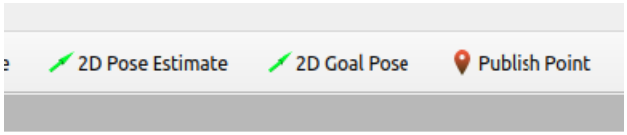


图 3-8 初始化位置

Nav2 Goal 是设置导航目标点的工具，通过菜单中的“Nav2 Goal”为机器人指定一个目标位置，使用鼠标光标设置目标位置和狗的最终角度，当单击一个点并继续移动鼠标光标时，显示的绿色长

箭头是角度设置器。

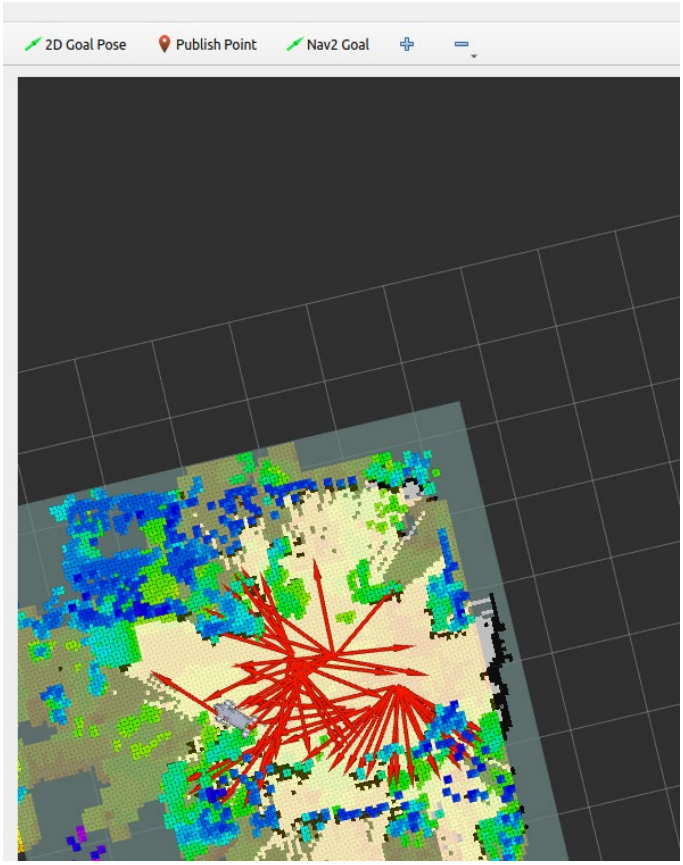


图 3-9 导航目标点设置

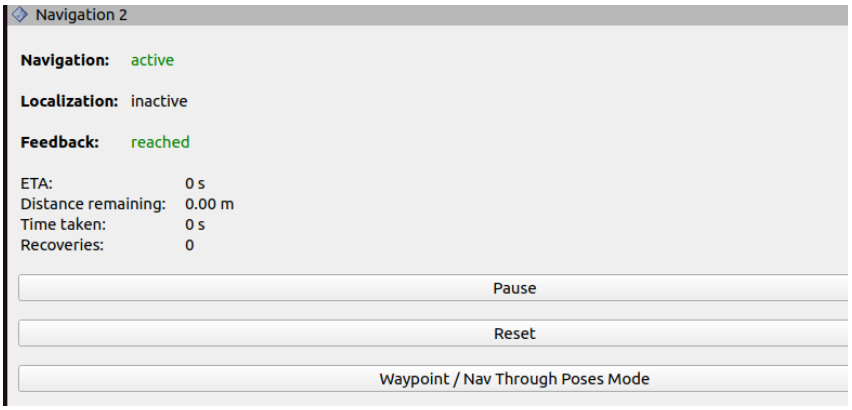


图 3-10 导航反馈

3.4.2、 命令行控制

可以通过命令行输入一个位姿信息，让机器人移动到指定导航点。

```
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose "{pose:{header: {frame_id: map}, pose:{position: {x: 1.607213, y: -0.10}}}}" --feedback
```

3.4.3、 代码控制 nav_to_pose.py

在 Navigation 2 中，提供了名为 nav2_simple_commander 的 Python 库，将常用的导航相关操作封装成相应的类，可以直接调用它来实现对导航的操作。

```
def main():
    rclpy.init()
    navigator = BasicNavigator()
    # 等待导航启动完成
    #navigator.waitUntilNav2Active()
    # 设置目标点坐标
    goal_pose = PoseStamped()
    goal_pose.header.frame_id = 'map'
    goal_pose.header.stamp = navigator.get_clock().now().to_msg()
```

```
        goal_pose.pose.position.x = 0.35
        goal_pose.pose.position.y = 0.14
        goal_pose.pose.orientation.w = 0.99
        # 发送目标接收反馈结果
        navigator.goToPose(goal_pose)
        while not navigator.isTaskComplete():
            feedback = navigator.getFeedback()
            navigator.get_logger().info(
                                                f'预计：
{Duration.from_msg(feedback.estimated_time_remaining).nanosecond
s / 1e9} s 后到达')
            # 超时自动取消
                if Duration.from_msg(feedback.navigation_time) >
Duration(seconds=600.0):
                    navigator.cancelTask()
        # 最终结果判断
        result = navigator.getResult()
        if result == TaskResult.SUCCEEDED:
```

```
navigator.get_logger().info('导航结果：成功')
```



图 3-11 导航反馈

3.5、 路点导航应用 waypoint_follower.py

3.5.1、 菜单操作

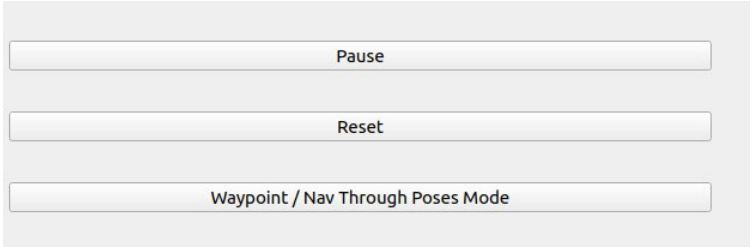


图 3-12 Waypoint 模式切换

切换到 Waypoint 模式，先使用 Nav2 Goal 依次设置多个路点，设置完成后，就可以开启路点导航了。在 RViz 的左下角的窗口上有三个按钮，单击最下面的 Start Waypoint Follwing 就可以启动路点导航。

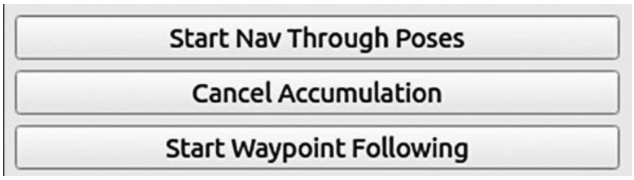


图 3-13 路点导航启动

3.5.2、 代码控制

使用之前在位姿获取代码中保存的位姿信息，为 Go2 机器人指定多个路点，实现路点导航。

```
    def main():
        rclpy.init()
        navigator = BasicNavigator()
        navigator.waitUntilNav2Active()
        # 创建点集
        goal_poses = []
        goal_pose1 = PoseStamped()
        goal_pose1.header.frame_id = 'map'
        goal_pose1.header.stamp =
navigator.get_clock().now().to_msg()
        goal_pose1.pose.position.x = 0.0
```

```
        goal_pose1.pose.position.y = 0.0
        goal_pose1.pose.orientation.w = 1.0
        goal_poses.append(goal_pose1)
        goal_pose2 = PoseStamped()
        goal_pose2.header.frame_id = 'map'
        goal_pose2.header.stamp =
navigator.get_clock().now().to_msg()
        goal_pose2.pose.position.x = 2.0
        goal_pose2.pose.position.y = 0.0
        goal_pose2.pose.orientation.w = 1.0
        goal_poses.append(goal_pose2)
        goal_pose3 = PoseStamped()
        goal_pose3.header.frame_id = 'map'
        goal_pose3.header.stamp =
navigator.get_clock().now().to_msg()
        goal_pose3.pose.position.x = 2.0
        goal_pose3.pose.position.y = 2.0
        goal_pose3.pose.orientation.w = 1.0
```

```
    goal_poses.append(goal_pose3)
    # 调用路点导航服务
    navigator.followWaypoints(goal_poses)
    # 判断结束及获取反馈
    while not navigator.isTaskComplete():
        feedback = navigator.getFeedback()
        navigator.get_logger().info(
            f'当前目标编号：{feedback.current_waypoint}')
    # 最终结果判断
    result = navigator.getResult()
    if result == TaskResult.SUCCEEDED:
        navigator.get_logger().info('导航结果：成功')
```

3.6、 构建导航服务节点

将上述导航的相关代码构建为导航服务节点，创建功能包 go2_application，构建类型为 ament_python。



图 3-14 导航功能包

在项目配置文件 setup.py 中指定节点服务入口。

```python
from setuptools import find_packages, setup

package_name = 'go2_navigation'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    description='TODO: Package description',
    license='TODO: License declaration',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'init_robot_pose=go2_application.init_robot_pose:main',
            'get_robot_pose=go2_application.get_robot_pose:main',
            'nav_to_pose=go2_application.nav_to_pose:main',
            'waypoint_follower=go2_application.waypoint_follower:main',
        ],
    },
```

图 3-15 导航节点设置

回到项目根目录，执行以下编译指令。

```
colcon builds
```

回到项目根目录 ros2_ws，通过以下指令启动导航节点提供的服

务。

```
source install/setup.bash

ros2 run go2_application get_robot_pose

ros2 run go2_application init_robot_pose
```

4、 项目总结

通过本项目的学习，我们可以将 Go2 机器人和 ROS2 平台紧密结合起来，创造出多样化且强大的机器人应用场景，包括但不限于单点导航/路点导航、物体追踪或自动巡逻等任务。